Project Title: 2D Cricket Web App

Course: Web Programming - B

Student Name: Humayun Bilal

Student ID: 23i-0832

Introduction

This project is a simple 2D cricket simulation built with React.
The user selects a batting style (Aggressive or Defensive), then presses Play Ball while a moving slider runs across the power bar.
The final outcome for each ball is decided using probability mapping, then displayed in the scoreboard, commentary panel, and match summary.

Game Logic and working mechanism

1. Match setup
   The game is limited by total overs/balls and total wickets.
   Player chooses a batting style: Aggressive or Defensive.

2. Probability-based outcome
   Each style has different probabilities for outcomes (W, 0, 1, 2, 3, 4, 6).
   When you press Play Ball, the current slider position is matched against cumulative probability ranges to decide the result.

3. Ball processing flow
   After Play Ball:

- Ball animation starts
- Bat swing animation triggers near contact time
- Outcome is calculated
- Runs/wickets/balls are updated
- Commentary text is updated
- Ball state resets for next delivery

4. Score update rules
   If outcome is W, wickets increase by 1.
   If outcome is a run value, that number is added to total runs.
   Balls bowled always increase by 1 each delivery.

5. Match end condition
   Game ends when either:

- All balls are completed, or
- All wickets are lost

Then final score and overs are shown as the match summary.

Probability Mapping Explanation

2.1 Core Idea

Each batting style has a probability table for outcomes:

- W (Wicket)
- 0 (Dot Ball)
- 1, 2, 3 runs
- 4 (Boundary)
- 6 (Six)

The slider position is a value between 0 and 1.
The app checks cumulative probability ranges from left to right and picks the first range that contains the slider position.

2.2 Mapping Logic (Concept)

If cumulative probability intervals are:

- 0.00 to 0.22 -> W
- 0.22 to 0.30 -> 0
- 0.30 to 0.46 -> 1
- etc.

Then a slider position of 0.35 maps to outcome 1.

2.3 Aggressive Mode Probabilities

- Wicket: 22%
- Dot Ball: 8%
- 1 Run: 16%
- 2 Runs: 14%
- 3 Runs: 5%
- Four: 20%
- Six: 15%

Aggressive mode has higher risk (more wickets) and higher reward (more boundaries).

2.4 Defensive Mode Probabilities

- Wicket: 8%
- Dot Ball: 24%
- 1 Run: 28%
- 2 Runs: 18%
- 3 Runs: 7%
- Four: 10%
- Six: 5%

Defensive mode is safer, with more singles/dot balls and fewer big shots.

3. Animation Explanation

3.1 Slider Animation

When the game is idle, the slider continuously moves left and right on the power bar.
This creates timing-based interaction where the user chooses when to press Play Ball.

3.2 Ball Travel Animation

After Play Ball is pressed, the ball visually moves from the bowler area to the batter area.
This movement represents delivery progress before the result is resolved.

3.3 Bat Swing Animation

Near the end of ball travel, the bat rotates briefly to simulate contact timing.
After contact, the bat returns to resting angle.

3.4 Sequencing and Timing
The game uses timed transitions:

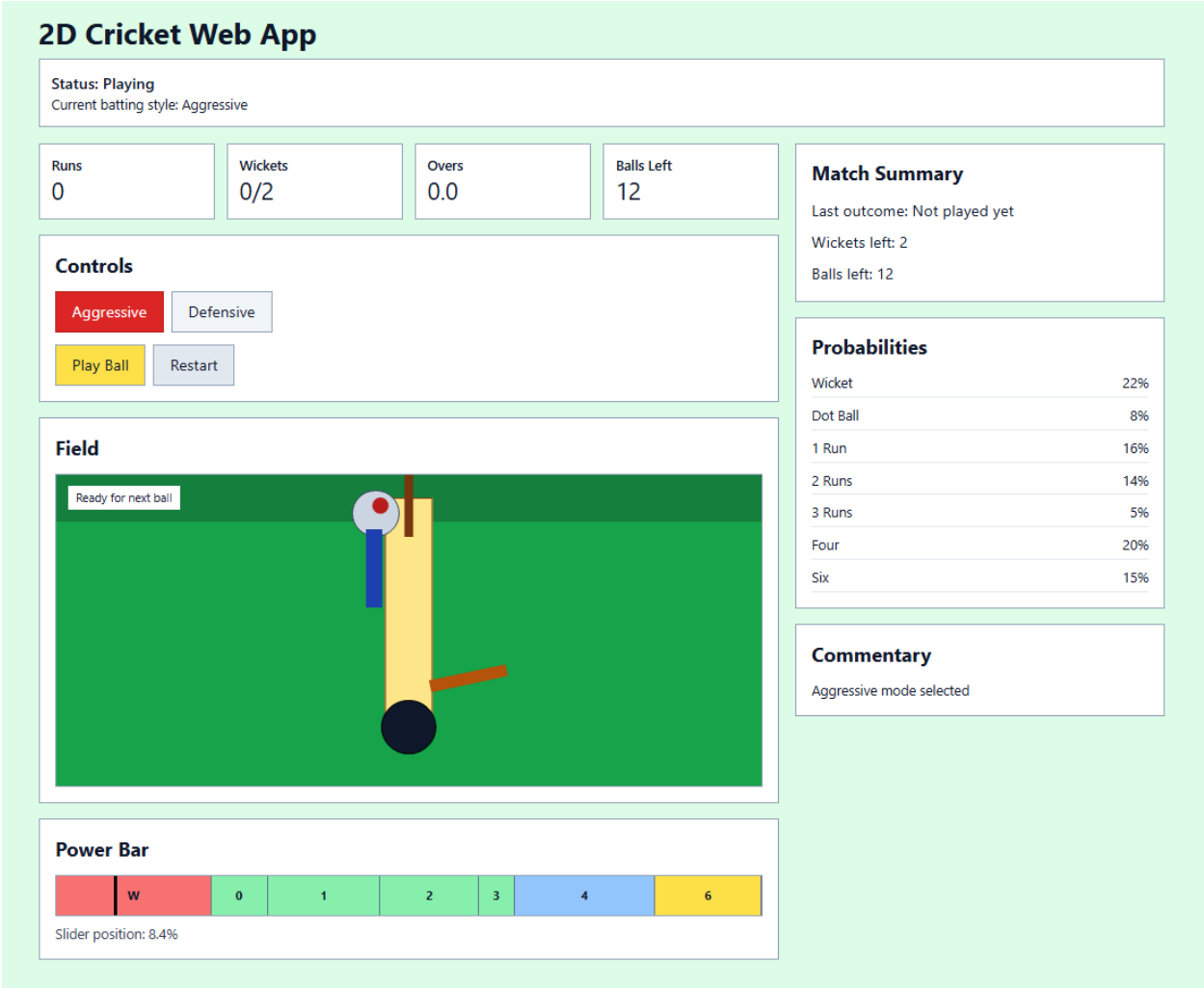
- Ball starts moving
- Bat swings near impact moment
- Outcome appears
- Ball state resets for next delivery

This gives a clear play cycle and makes interaction feel realistic for a mini 2D cricket simulation.

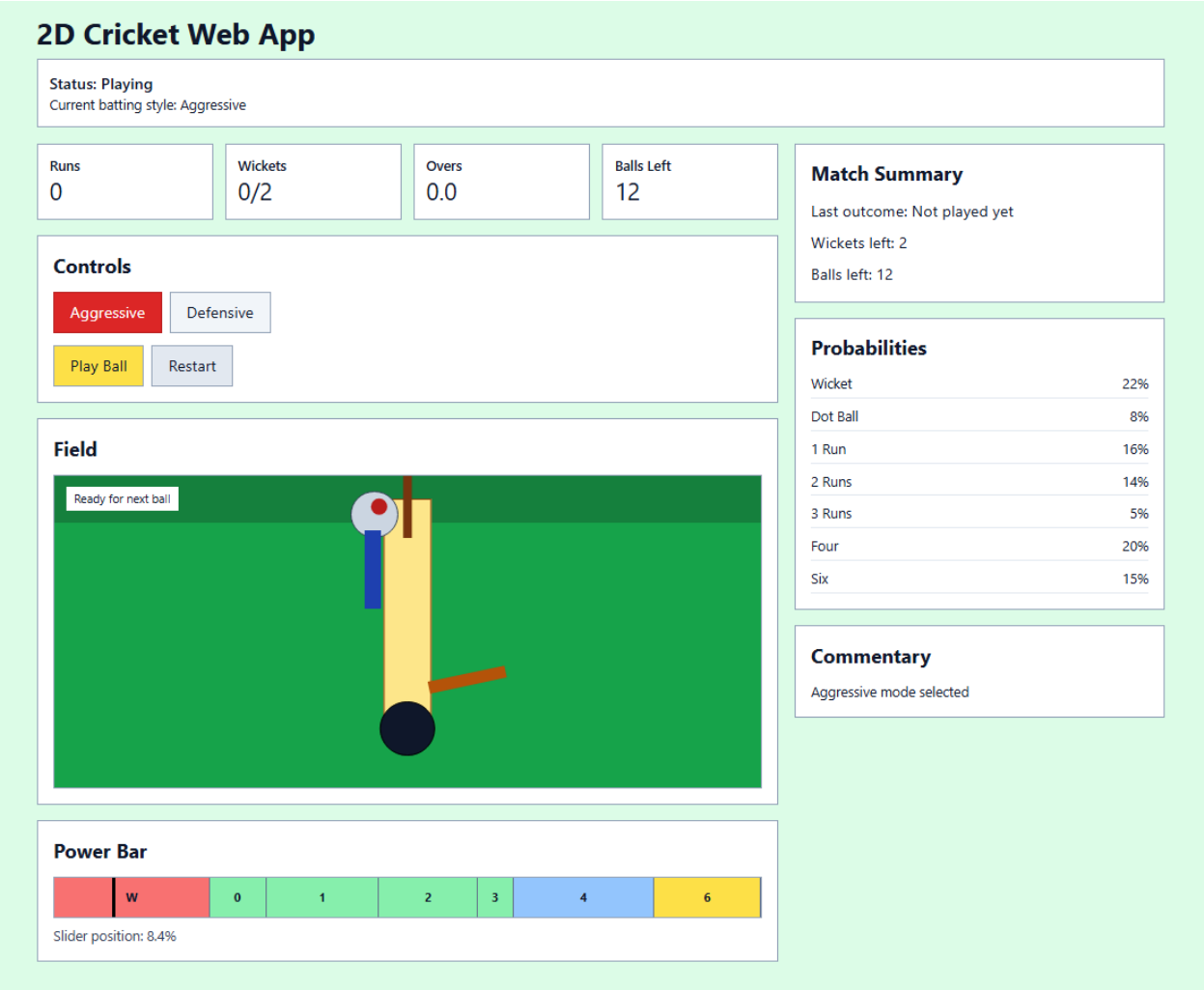
4. **Screenshots**

5. Important note for submission:
   Each screenshot should include a visible sticky note showing your name and date for authenticity.

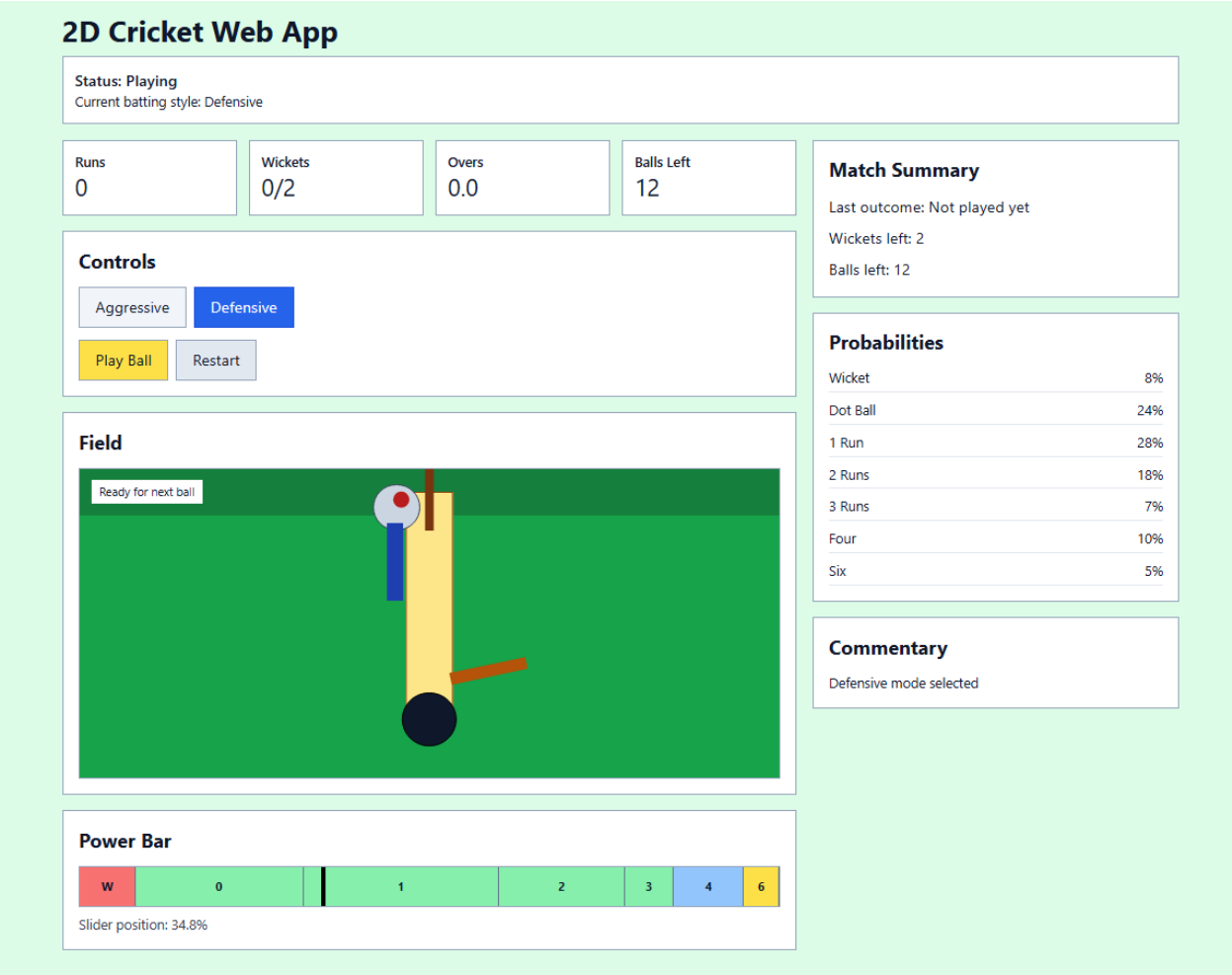
Screenshot 1: Full Game Interface (Initial State)



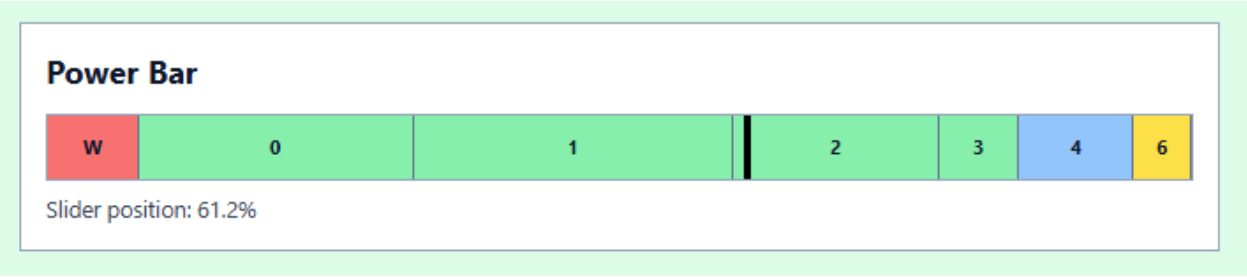
Screenshot 2: Aggressive Mode Selected + Probability Table Visible



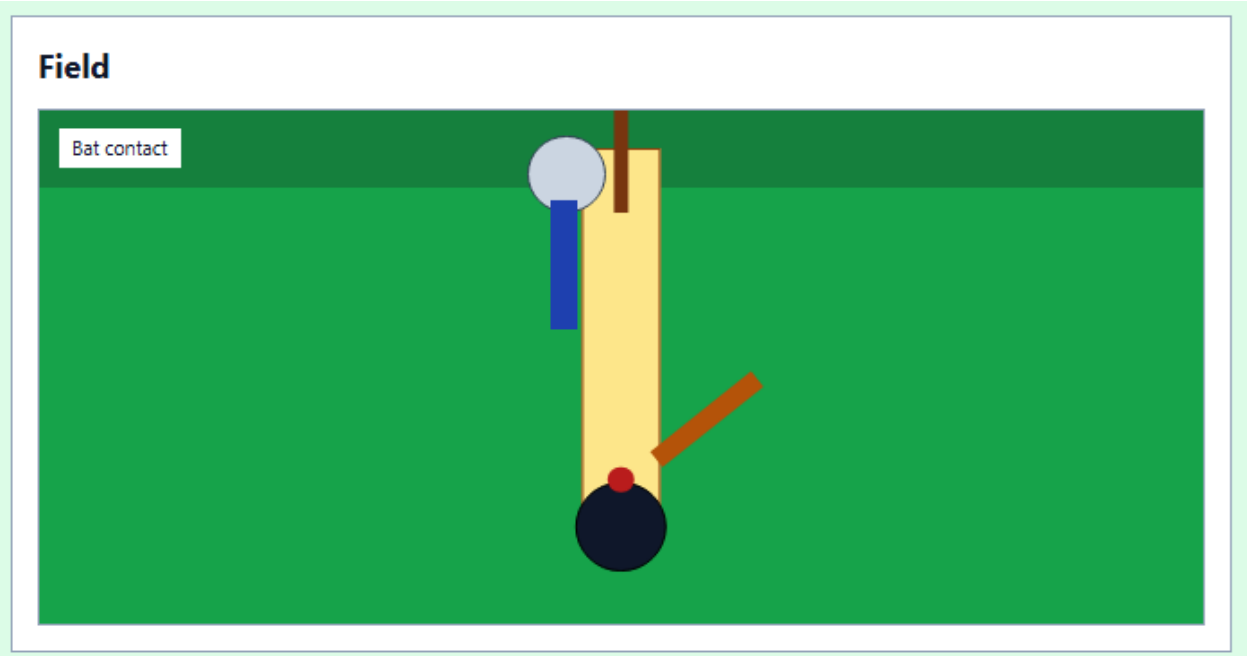
Screenshot 3: Defensive Mode Selected + Probability Table Visible



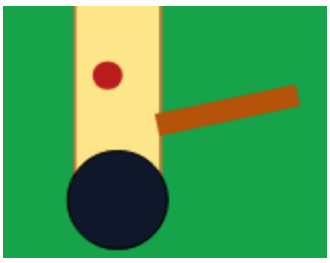
Screenshot 4: Slider Position During Play (Before Clicking Play Ball)



Screenshot 5: Ball Movement Animation in Field View

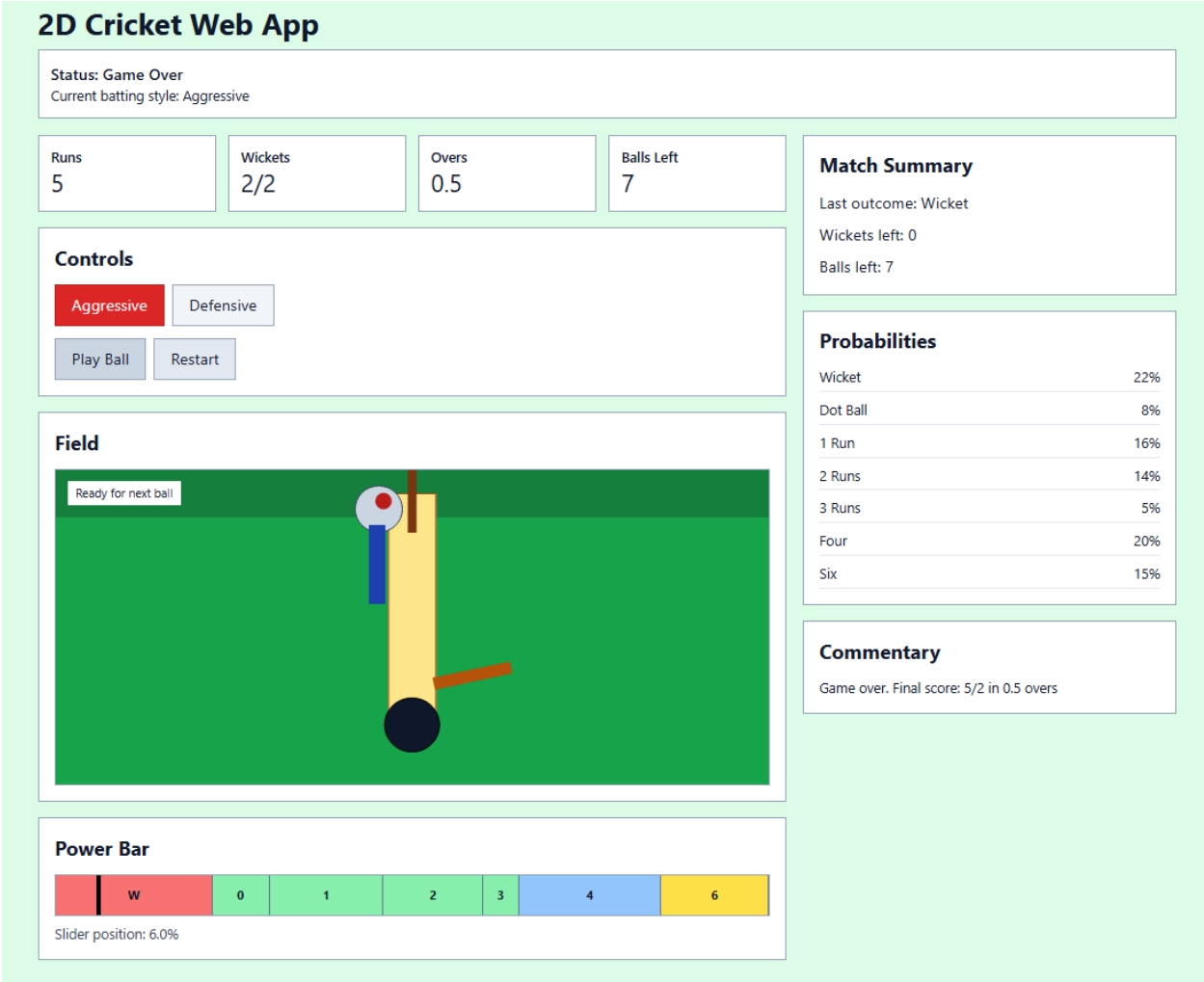


Screenshot 6: Bat Swing / Contact Moment



Screenshot 7: Commentary and Score Update After a Ball

Screenshot 8: Match End Screen (Final Score and Overs)



Conclusion

This project demonstrates probability-driven game behavior and simple UI animation sequencing in a React web application.

The probability mapping differentiates Aggressive and Defensive play styles, while slider, ball, and bat animations improve user engagement and clarity of the game flow.